

MASTERSEMINAR · PITCH · SS 2026

Optimierung von Transferfunktionen im Volume Rendering mit Reinforcement Learning

Kevin Kunkel

Universität Leipzig · Betreuer: Prof. Gerik Scheuermann, Dr. Thomas Wischgoll, Dr. Baldwin Nsonga



UNIVERSITÄT
LEIPZIG

- | | |
|---|--------|
| 1 — Motivation: Warum Transferfunktions-Design so mühsam ist | 2 Min. |
| 2 — Verwandte Arbeiten & Fragestellung | 2 Min. |
| 3 — Architektur: Sprache/Text → Kommando → Render → Bewertung | 3 Min. |
| 4 — Die Transferfunktion: 24 Werte, echte Hounsfield-Einheiten | 2 Min. |
| 5 — Der Prototyp: Live-Demo der Chat-Oberfläche | 2 Min. |
| 6 — Kommandogrammatik & Parser-Auswertung | 2 Min. |
| 7 — Ergebnisse an echten CT-Daten | 1 Min. |
| 8 — Erkenntnisse aus dem Testen | 1 Min. |
| 9 — Von der Baseline zum RL-Agenten: SAC, Online-Lernen, RLHF | 4 Min. |

Direct Volume Rendering macht volumetrische Daten (CT, MRT, Simulationen) über eine **Transferfunktion** sichtbar.

Definition: Transferfunktion $T : [\min, \max] \rightarrow [0, 1]^4$ — jeder Skalarwert bekommt Farbe (RGB) und Opazität (α). Die Wahl von T entscheidet, welche Strukturen sichtbar werden.

Das Problem:

- Parameterraum groß, unintuitiv
- Erfordert Domänenwissen (z. B. Hounsfield-Einheiten)
- Manuelles Nachjustieren kostet Zeit

Der Wunsch:

- “Zeig mir nur den Knochen”
- “Weichgewebe ist zu dominant”
- Sprache/Text statt Schieberegler

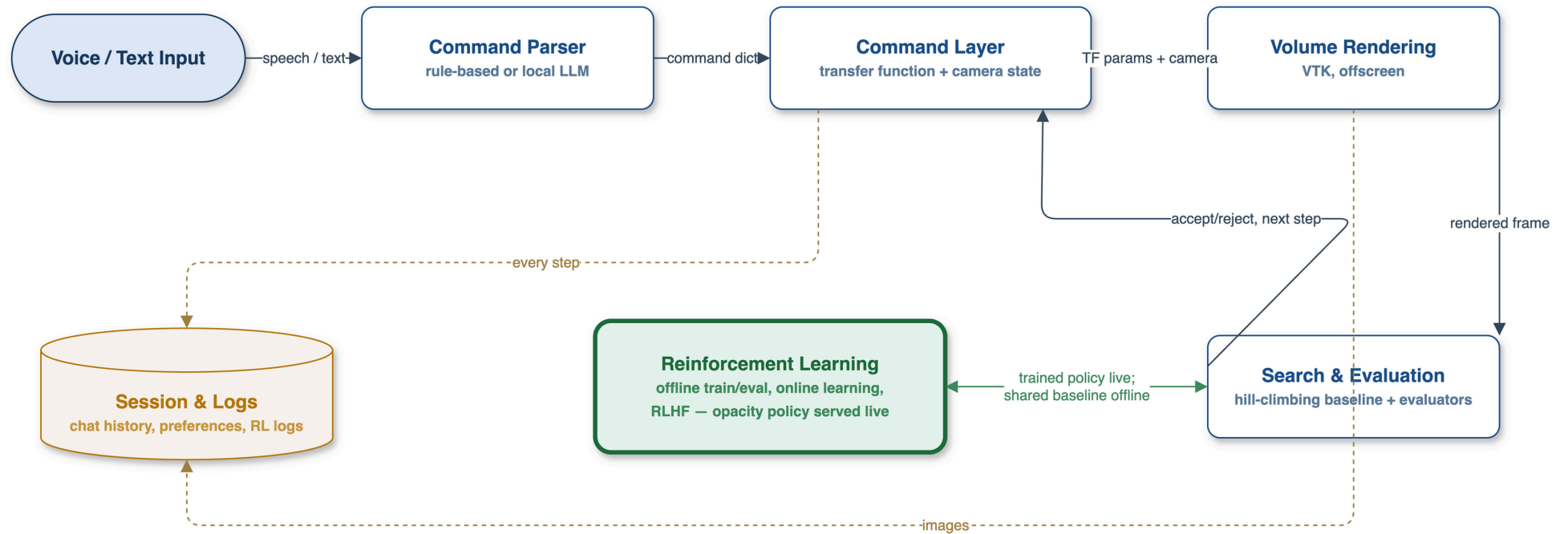
TF-Design: sprachbasiert & nicht

- T2TF: Text $\rightarrow$ TF, differenzierbar (Jeong et al., 2024)
- IntuiTF: MLLM als Explorer **und** Bewerter (Wang et al., 2025)
- NLI4VolVis: Chat-Agenten für Volume-Viz (Ai et al., 2025a)
- Segmentierung via Vision-Transformer (Engel et al., 2024)
- TF-Optimierung über Bildvergleich (Neuhäuser & Westermann, 2023)

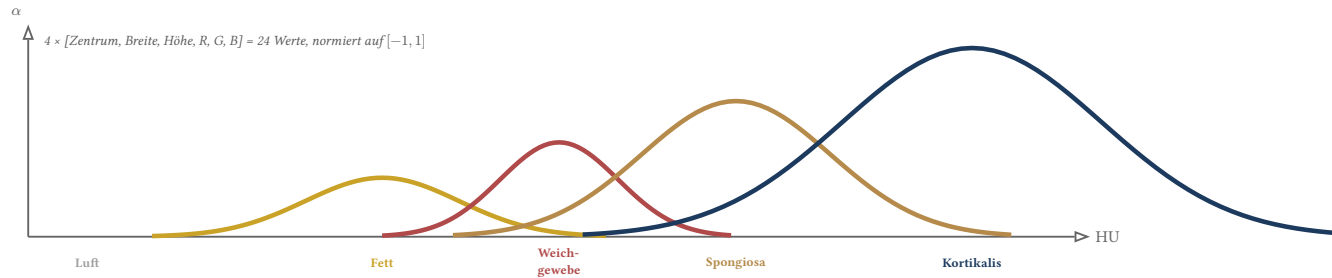
RL & Evaluation für Parameterexploration

- Deep RL mit Menschen im Loop (Scurto et al., 2021)
- RL für sprachgesteuerte Viewpoint-Navigation (Zhao & Tao, 2025)
- Harte, reproduzierbare Metriken statt MLLM-Urteil (Ai et al., 2025b)

Fragestellung Lässt sich Transferfunktions-Design durch Sprache **und** eine exakte, kostenlose Metrik (statt eines MLLM-Richters) so weit automatisieren, dass daraus eine trainierbare RL-Baseline entsteht?



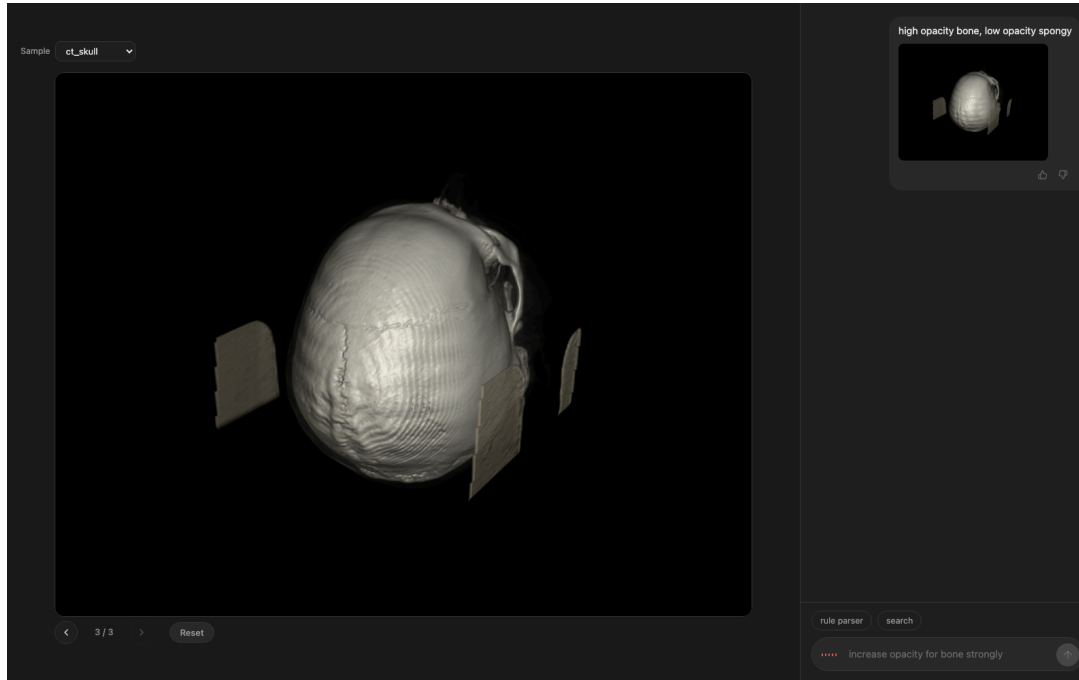
Die Transferfunktion: 24 Werte



$$\alpha(x) = \text{clip} \left(\sum_{i=1}^4 h_i \cdot \exp \left(-\frac{1}{2} \left(\frac{x - c_i}{w_i} \right)^2 \right), 0, 1 \right)$$

Design-Entscheidung: Zentren c_i und Breiten w_i liegen in echten **Hounsfield-Einheiten** (HU) — nicht in $[0, 255]$ wie bei den meisten TF-Editoren.

Konsequenz: Dieselbe Gewebetabelle (Luft, Fett, Weichgewebe, Spongiosa, Kortikalis) funktioniert unverändert auf synthetischem, CT- **und** MRT-Material (linear auf denselben HU-Bereich reskaliert).



Lokal, offline-first:

- Chat-Oberfläche (FastAPI + Vanilla JS)
- Text **und** Sprache (faster-whisper, lokal)
- Regel- **und** LLM-Parser (Ollama, lokal)
- Sofortiges Rendering nach jedem Kommando

Für RL-Daten von Anfang an gebaut:

- Jeder Schritt, jedes Bild, jede LLM-Anfrage wird geloggt
- 👍/👎-Feedback pro Antwort
- Verzweigbare Historie (zurück & neu verzweigen)

```
increase|decrease opacity for <gewebe> [slightly|moderately|strongly]  
show only <gewebe> [and <gewebe> ...]  
<low|medium|high> opacity for <gewebe> [, ... weitere Gewebe]  
reset
```

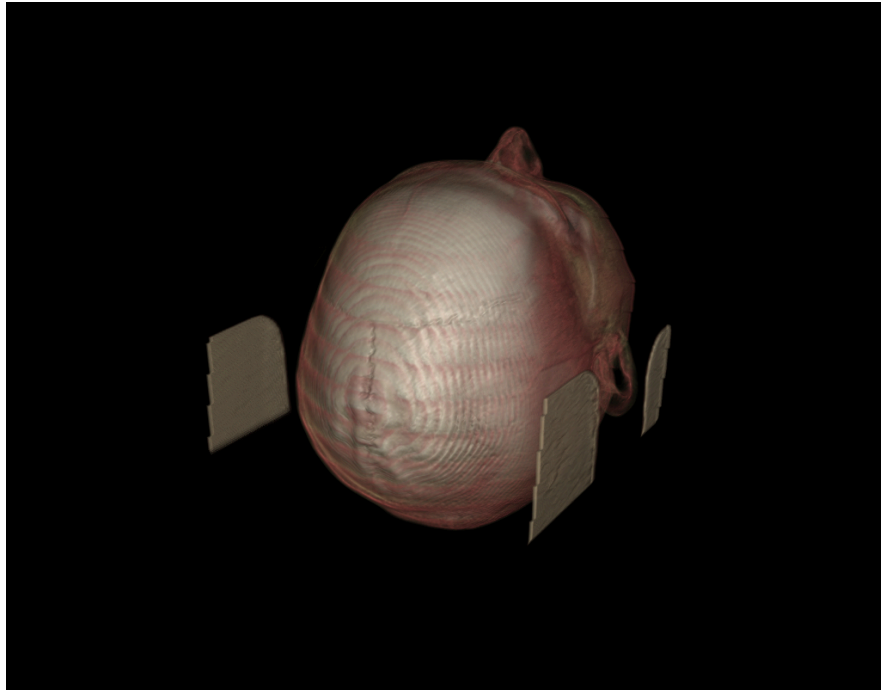
Testset: 20 freie Formulierungen, die der Regelparser **per Design** nicht versteht (z. B. “*make the skeleton pop*”, “*the spongy core is barely there*”).

Der LLM-Parser bekommt dieselbe Synonymtabelle wie der Regelparser im Systemprompt — das behebt einen systematischen Fehler (Spongiosa → generisches “Bone”).

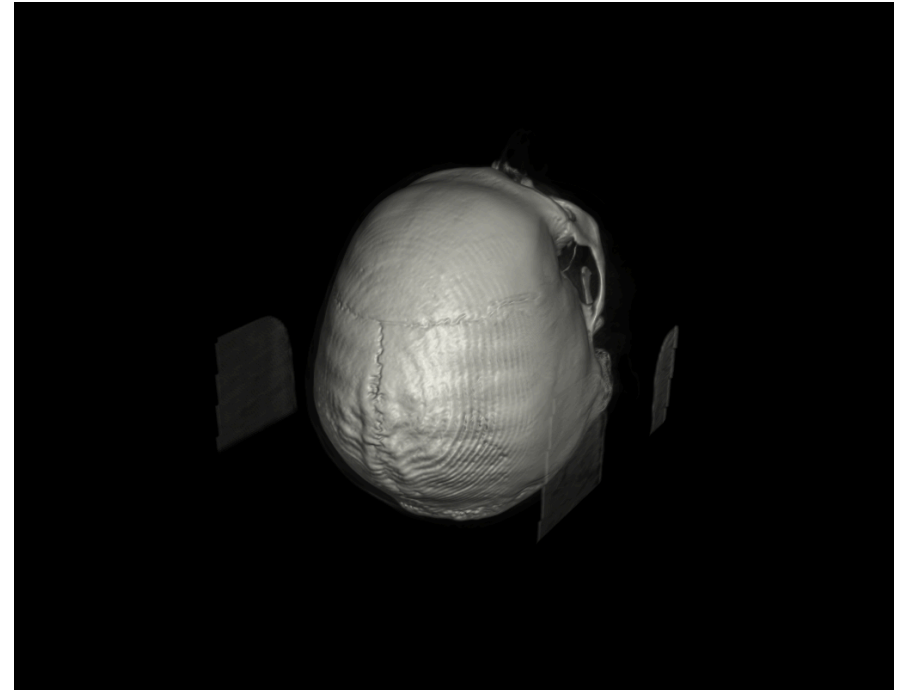
Parser	Korrekt
Regelbasiert	0 / 20
LLM (qwen2.5:7b)	18 / 20

Ergebnisse an echten CT-Daten

Reale, de-identifizierte CT-Datensätze (3D Slicer Testdaten) — kein zusätzlicher Code für echte Daten nötig, da bereits in HU gearbeitet wird.



“Startzustand” — Standard-TF



“show only bone” — ein Kommando

Erkenntnisse aus dem Testen

Drei konkrete Bugs, gefunden durch tatsächliches Testen an echten Daten und echter Nutzung
— nicht nur durch Unit-Tests:

Render-Crash Rendering nahm implizit einen würfelförmigen Datensatz an — stürzte sofort ab auf echten $512 \times 512 \times 139$ -CT-Daten. Fix: Dimensionen pro Achse statt einer einzigen Kantenlänge.

Sättigungs-Bug “increase opacity for bone strongly” erreichte in **einem** Schritt das Maximum — jedes weitere Kommando war ein stiller No-Op. Fix: Schrittweite proportional zur verbleibenden Distanz zum Rand statt fixer Betrag.

Spracherkennung Whisper (small) verhält sich bei “spongy” reproduzierbar — “sponges”, “spudgy”, “spore G”. In 24 echten Sprachkommandos scheiterten 19 (79 %) am Parser, nicht am System.

Der SAC-Agent (stable-baselines3) ist implementiert, trainiert und gegen die Baseline evaluiert – online gegen opacity_mass, ganz ohne menschliche Trainingsdaten.

Zustand TF-Vektor (24) + Ziel-Gewebe + Richtung + mass_fraction (31-dim)

Aktion kontinuierliches Delta auf die Höhe des aufgelösten Ziel-Peaks

Reward $r = \text{sign}(d) \cdot (\varphi(p', t) - \varphi(p, t)) - \Delta \text{mass_fraction}$ im Ziel-Band

Ergebnis **0.344** vs. **0.347** (Baseline) mass_fraction, 20 Test-Episoden

Anmerkung Nahezu gleichauf mit dem Hill-Climber, aber langsamer konvergent (3.3 vs. 2.7 Schritte bis 90 %) – ein optimierter Hand-Regler ist auf diesem niedrigdimensionalen Problem ein starker Gegner (Zhao & Tao, 2025). Mittlerweile live im Chat-UI nutzbar (server.py, policy-Toggle) – als eingefrorene Inferenz, noch ohne Nachlernen aus echter Nutzung (siehe nächste Folien).

Statt eines einzigen 200k-Batch-Trainings: ein frischer Agent lernt fortlaufend auf einer einzelnen Umgebung, alle 5000 Schritte gegen dieselben 20 Test-Episoden neu bewertet (rl/online_train.py).

Budget	50 000 Schritte, eine Umgebung statt vier parallel
Verlauf	0.199 (5k) $\rightarrow$ 0.347 (15k) $\rightarrow$ 0.346 (50k) mass_fraction
Konvergenz	Schritte-bis-90% fällt von 17.4 auf 3.2
Budget-Anteil	nur 25 % der Umgebungsschritte des Offline-Laufs (200k)

Anmerkung Landet mit nur einem Viertel des Offline-Budgets fast exakt zwischen Policy- (0.344) und Hill-Climb-Baseline (0.347) — die Lernkurve konvergiert klar, nicht nur zufällig. Zeigt: dasselbe SAC-Setup lernt auch als fortlaufender Prozess, nicht nur als einmaliger Batch-Lauf — Voraussetzung für Nachlernen aus echtem Feedback (nächste Folie).

mass_fraction misst nur, ob sich die TF-Kurve wie gewünscht bewegt hat — nie, ob das gerenderte Bild tatsächlich gut aussieht. Fünf neue Module schließen diese Lücke (Scurto et al., 2021): ein kleines Reward-Modell, gelernt aus menschlichen Besser/Schlechter-Urteilen über gerenderte Bildpaare.

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \sigma(R_\theta(z_i)) + (1 - y_i) \log(1 - \sigma(R_\theta(z_i)))], \quad \tilde{r}(z) = 2\sigma(R_\theta(z)) - 1$$

Label-Signal	50 gerenderte Paare gezielt (collect_preferences.py) oder passiv aus echter Chat-Nutzung (ingest_feedback.py, aus out/feedback.jsonl)
Reward-Modell	10 → 16 → 1 MLP auf Bildmerkmalen (Mean/Std/Coverage/Entropy)
Echter Lauf	15 nutzbare Labels aus echter Nutzung → train_acc 0.833 , val_acc 1.000 (n=12/3 — zu klein für eine echte Aussage)
RL-Umgebung	RewardModelTFEnv: ein Render pro Schritt, 109 ms gemessen; RL-Training gegen das Reward-Modell selbst noch nicht gelaufen

Anmerkung Passive Datensammlung (`ingest_feedback.py`) statt dedizierter Session war die bessere Idee: nutzt Features, die `server.py` beim normalen Rendern ohnehin schon loggt — skaliert mit echter Nutzung statt einer Sondersitzung. `n=15` ist ein Pilot, kein belastbares Ergebnis — bewusst so kommuniziert.

Vielen Dank für eure Aufmerksamkeit!

Fragen?

- Ai, K., Miao, H., Li, Z., Wang, C., & Liu, S. (2025b). An Evaluation-Centric Paradigm for Scientific Visualization Agents. *Arxiv Preprint Arxiv:2509.15160*.
- Ai, K., Tang, K., & Wang, C. (2025a). NLI4VolVis: Natural Language Interaction for Volume Visualization via LLM Multi-Agents and Editable 3D Gaussian Splatting. *Arxiv Preprint Arxiv:2507.12621*.
- Engel, D., Sick, L., & Ropinski, T. (2024). Leveraging Self-Supervised Vision Transformers for Segmentation-based Transfer Function Design. *IEEE Transactions on Visualization and Computer Graphics*. <https://doi.org/10.1109/TVCG.2024.3401755>
- Jeong, S., Li, J., Johnson, C., Liu, S., & Berger, M. (2024). Text-based transfer function design for semantic volume rendering. *Arxiv Preprint Arxiv:2406.15634*.
- Neuhäuser, C., & Westermann, R. (2023). Transfer Function Optimization for Comparative Volume Rendering. *Computer Graphics Forum*, 42(7).
- Scurto, H., Van Kerrebroeck, B., Caramiaux, B., & Bevilacqua, F. (2021). Designing Deep Reinforcement Learning for Human Parameter Exploration. *ACM Transactions on Computer-Human Interaction*, 28(1), 1–35.
- Wang, Y., Pan, B., Wang, K., Liu, H., Mao, J., Liu, Y., Zhu, M., Huang, X., Chen, W., Zhang, B., & Chen, W. (2025). IntuiTF: MLLM-Guided Transfer Function Optimization for Direct Volume Rendering. *Arxiv Preprint Arxiv:2506.18407*.
- Zhao, X., & Tao, J. (2025). Natural Language-Driven Viewpoint Navigation for Volume Exploration via Semantic Block Representation. *Arxiv Preprint Arxiv:2508.06823*.